Le Stringhe in C

Dichiarazione, input/output, lunghezza e accesso ai caratteri

Alberto Testa

PROGRAMMAZIONE IN C

Cosa è una Stringa in C?

Definizione fondamentale

In C, una **stringa** è semplicemente un **array di char** terminato dal carattere speciale `'\0'` (null terminator). Non esiste un tipo nativo `string` come in altri linguaggi: tutto si costruisce su array e puntatori.

Il terminatore `'\0'` ha valore ASCII 0 e segnala la fine logica della stringa. Senza di esso, le funzioni di libreria non sapranno dove fermarsi.

Schema visivo: `char s[] = "Ciao"`

Indice	0	1	2	3	4
Valore	'C'	'i'	'a'	'o'	'\0'

⚠️ **Lunghezza logica**: 4 caratteri (quelli visibili)
Dimensione array: 5 celle (include `'\0'`)

Ricorda: devi sempre riservare **un byte in più** per il terminatore quando dichiari un array di stringhe.

Dichiarazione e Inizializzazione

Esistono quattro modi principali per dichiarare e inizializzare una stringa in C. Ognuno ha caratteristiche e casi d'uso specifici.

1

Dimensione automatica

```
char s[] = "Ciao";
```

Il compilatore calcola automaticamente la dimensione (5 byte: 4 caratteri + `'\0'`). Metodo più semplice e sicuro.

2

Array sovradimensionato

```
char s[10] = "Ciao";
```

Si riserva più spazio del necessario. Le celle da 5 a 9 rimangono disponibili per future modifiche o estensioni della stringa.

3

Carattere per carattere

```
s[0]='C'; s[1]='i';
s[2]='a'; s[3]='o';
s[4]='\0';
```

Assegnazione manuale cella per cella. **Fondamentale**: non dimenticare mai `'\0'` finale!

4

Solo dichiarazione

```
char s[10];
/* poi riempimento
   con ciclo o input */
```

Si dichiara lo spazio e si riempie in seguito, ad esempio leggendo da input utente. Più flessibile, ma richiede attenzione.

Output di Stringhe

Per stampare stringhe a schermo si usano tre funzioni principali, ciascuna con comportamento diverso. Scegliere quella giusta rende il codice più leggibile e corretto.

Funzione	Sintassi	Comportamento	Nota
`printf`	`printf("%s", s);`	Stampa la stringa	Nessun `\n` automatico
`puts`	`puts(s);`	Stampa la stringa	Aggiunge `\n` automatico
`putchar`	`putchar(s[i]);`	Stampa un solo char	Utile nei cicli

```
/* printf */
char s[] = "Ciao";
printf("Messaggio: %s\n", s);

/* puts */
puts(s); /* equivale a printf + \n */

/* putchar in ciclo */
int i = 0;
while (s[i] != '\0') {
    putchar(s[i]);
    i++;
}
```

Quando usare cosa? Usa `printf` quando devi formattare output complesso. Usa `puts` per stampe veloci di intere stringhe. Usa `putchar` quando elabori la stringa carattere per carattere.

Input di Stringhe: `scanf`

Come funziona

`scanf("%s", s)` legge caratteri fino al primo **spazio, tab o newline**. Non legge frasi intere, ma è veloce per token singoli come codici o username.

```
char codice[21];
printf("Inserisci codice: ");
scanf("%20s", codice);
printf("Codice: %s\n", codice);
```

Quando usarlo

- Codici prodotto o seriali
- Username e token senza spazi
- Valori numerici in forma testuale

⚠️ Rischio Buffer Overflow

Senza modificatore di larghezza, `scanf("%s", s)` può scrivere oltre i limiti dell'array se l'utente inserisce troppi caratteri.

```
/* SBAGLIATO - overflow possibile */
char s[10];
scanf("%s", s);

/* CORRETTO - al massimo 9 char */
char s[10];
scanf("%9s", s);
```

Il numero nel formato indica il **massimo di caratteri da leggere** (escluso `'\0'`): usa sempre `N-1` rispetto alla dimensione dell'array.

Input di Stringhe: `fgets`

`fgets` è la funzione più sicura per leggere stringhe da tastiera perché gestisce automaticamente il limite di lunghezza e supporta gli spazi.

Sintassi e utilizzo

```
char nome[50];
printf("Inserisci nome: ");
fgets(nome, 50, stdin);
```

`fgets(s, dimensione, stdin)` legge **al massimo dimensione-1 caratteri**, inclusi spazi, e termina con `'\0'`.

Quando usarlo

- Nomi e cognomi con spazi
- Indirizzi e testo libero
- Qualsiasi input con spazi

⚠️ Problema: include `'\n'`

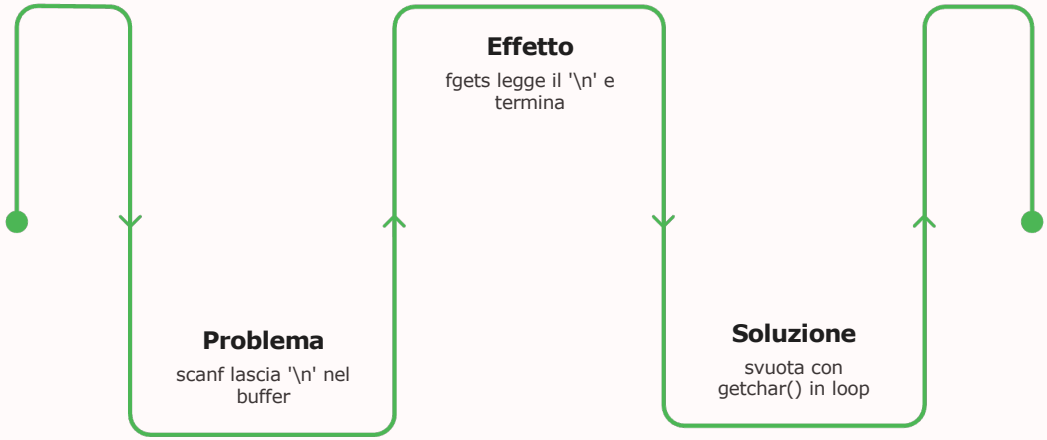
`fgets` include il carattere di fine riga nella stringa. Va rimosso manualmente con questo schema:

```
int i = 0;
while (s[i] != '\n' &&
       s[i] != '\0') {
    i++;
}
s[i] = '\0';
```

Il ciclo avanza finché non trova `'\n'` o `'\0'`, poi sovrascrive il carattere trovato con `'\0'`, troncando efficacemente la stringa.

Problema del Buffer Residuo

Uno dei bug più frequenti nei programmi C: mischiare `scanf` e `fgets` senza svuotare il buffer di input causa comportamenti inattesi.



❌ Codice con il bug

```
int eta;
char nome[50];
scanf("%d", &eta);
fgets(nome, 50, stdin);
/* nome rimane vuoto! */
```

✅ Soluzione: svuotare il buffer

```
int eta;
char nome[50];
scanf("%d", &eta);
/* svuota buffer */
int c;
while ((c = getchar()) != '\n'
 && c != EOF);
fgets(nome, 50, stdin);
/* ora funziona correttamente */
```

Calcolo Manuale della Lunghezza

Algoritmo con ciclo `while`

Per calcolare la lunghezza di una stringa, si scorre l'array fino a trovare il terminatore `'\0'`. L'indice raggiunto in quel momento coincide esattamente con il numero di caratteri validi.

```
int lung = 0;
while (s[lung] != '\0') {
    lung++;
}
/* lung ora contiene la lunghezza */
printf("Lunghezza: %d\n", lung);
```

Intuizione chiave: `'\0'` si trova sempre all'indice pari alla lunghezza. Per "Ciao" (4 char), il terminatore è all'indice 4.

Traccia dell'algoritmo su "Ciao"

Iterazione	`s[lung]`	Condizione	`lung`
0	'C'	≠ '\0' → vera	1
1	'i'	≠ '\0' → vera	2
2	'a'	≠ '\0' → vera	3
3	'o'	≠ '\0' → vera	4
4	'\0'	= '\0' → falsa	**4** ✓

Anticipazione: `strlen()`

La libreria `<string.h>` fornisce `strlen(s)` che esegue esattamente questo algoritmo. Nei corsi avanzati la userete spesso, ma capire il meccanismo interno è fondamentale.

Accesso e Modifica per Indice

Operazioni di base

```
char s[] = "Ciao";

/* Lettura */
char c = s[0]; /* c = 'C' */

/* Modifica */
s[0] = 'X';    /* s = "Xiao" */

/* Troncamento */
s[2] = '\0';   /* s = "Xi" */
```

Conversione maiuscole ↔️ minuscole

Maiuscole e minuscole differiscono di esattamente **32 unità ASCII**:

```
/* min → MIN */
s[i] = s[i] - 32;
/* MIN → min */
s[i] = s[i] + 32;
```

Valori ASCII fondamentali

Categoria	Da	A
Cifre (0–9)	48	57
Maiuscole (A–Z)	65	90
Minuscole (a–z)	97	122

Per classificare un carattere si confronta il suo valore numerico con questi intervalli:

```
if (s[i] >= 65 && s[i] <= 90)
    /* è una lettera maiuscola */
if (s[i] >= 48 && s[i] <= 57)
    /* è una cifra */
```

In alternativa si può usare `s[i] >= 'A' && s[i] <= 'Z'`, più leggibile e equivalente.

Pattern di Scansione con `while`

Questo è il **pattern fondamentale** per qualsiasi elaborazione su stringhe in C. Bisogna saperlo scrivere a memoria e riconoscerlo immediatamente.

```
int i = 0;
while (s[i] != '\0') {
    /* elabora s[i] */
    i++;
}
```

🔢 Conteggio vocali

```
int voc = 0;
int i = 0;
while (s[i] != '\0') {
  if (s[i]=='a'||s[i]=='e'||
      s[i]=='i'||s[i]=='o'||
      s[i]=='u')
    voc++;
  i++;
}
```

🔍 Ricerca carattere

```
char cerca = 'z';
int trovato = 0;
int i = 0;
while (s[i] != '\0') {
  if (s[i] == cerca) {
    trovato = 1;
    break;
  }
  i++;
}
```

🔄 Sostituzione

```
int i = 0;
while (s[i] != '\0') {
 if (s[i] == 'a')
 s[i] = 'A';
 i++;
}
/* sostituisce tutte le
 'a' con 'A' */
```

Esempi Pratici (1/2)

APPLICAZIONI REALI

📋 Registro Presenze Scolastico

Lettura del nome studente con `fgets`, verifica che il campo non sia vuoto (solo `'\n'`), rimozione del terminatore di riga e stampa del log formattato con ora e nome.

📱 Messaggistica SMS

Limite di 160 caratteri: conteggio manuale con ciclo `while` senza ricorrere a `strlen`, visualizzazione barra di utilizzo percentuale e avviso se il messaggio supera il limite.

📦 Inserimento Prodotto a Magazzino

Combinazione di `scanf` per il codice prodotto (nessuno spazio) e `fgets` per la descrizione (testo libero). Validazione che i campi obbligatori non siano vuoti prima di salvare.

🖥️ Log di Sistema

Composizione di una riga di log con campi misti: timestamp numerico, livello di severità (stringa breve) e messaggio descrittivo. Uso combinato di `printf` con più specificatori di formato.

Esempi Pratici (2/2)

🔢 Conta Occorrenze

Ricerca di una lettera in un testo con spazi, con distinzione tra maiuscole e minuscole. Utilizza il pattern base di scansione con contatore.

🗣️ Distribuzione Vocali

Analisi fonetica: conta ogni vocale separatamente e mostra una barra grafica proporzionale per ciascuna, utile per statistiche linguistiche.

🔐 Validazione Password

Verifica 5 requisiti di sicurezza con flag booleani: lunghezza minima, cifra presente, maiuscola, minuscola, carattere speciale.

📊 Analisi Testo

Conteggio parole con variabile di stato (dentro/fuori parola), più statistiche complete: caratteri totali, spazi, cifre, lettere.

🏦 Verifica IBAN

Controllo formale per posizione: primi 2 char lettere (paese), poi 2 cifre di controllo, poi caratteri alfanumerici. Verifica strutturale senza algoritmo checksum.

🚫 Censura Testo

Sostituzione di una parola vietata con asterischi `*` direttamente nell'array (modifica in-place), scorrendo carattere per carattere con confronto sequenziale.

Errori Comuni da Evitare

Questi sono i cinque errori più frequenti tra chi inizia a programmare stringhe in C. Riconoscerli e saperli correggere è fondamentale.

1 Dimenticare `'\0'`

❌ `char s[4] = "Ciao";` — array troppo piccolo, overflow garantito.
✅ `char s[5] = "Ciao";` — spazio per il terminatore.

2 `scanf` senza limite di larghezza

❌ `scanf("%s", s);` — nessun controllo sulla lunghezza input.
✅ `scanf("%9s", s);` — per un array da 10 elementi.

3 `fgets` dopo `scanf` senza svuotare il buffer

❌ Chiamare `fgets` subito dopo `scanf` senza ciclo `getchar()`. Il `'\n'` residuo viene letto immediatamente da `fgets`.

4 Non rimuovere `'\n'` dopo `fgets`

❌ Usare la stringa letta con `fgets` senza rimuovere il `'\n'` finale, causando errori di confronto e stampa.

5 Confrontare stringhe con `==`

❌ `if (s == "Ciao")` — confronta indirizzi di memoria, non contenuti.
✅ Confronta carattere per carattere con un ciclo, oppure usa `strcmp()` da `<string.h>`.

Riepilogo: Le Stringhe in C

Ecco una mappa completa di tutti gli strumenti visti. Tienila come riferimento rapido durante i tuoi esercizi di programmazione.

Operazione	Strumento	Note importanti
Output	`printf`, `puts`, `putchar`	`puts` aggiunge `'\n'` automatico; `putchar` per un singolo carattere
Input senza spazi	`scanf("%Ns", s)`	N = limite caratteri (usa sempre il modificatore di larghezza!)
Input con spazi	`fgets(s, N, stdin)`	Rimuovere `'\n'` con ciclo; svuotare buffer se preceduto da `scanf`
Lunghezza	Ciclo `while` fino a `'\0'`	Oppure `strlen()` da `<string.h>`
Accesso carattere	`c = s[i]`	Lettura e scrittura tramite indice intero
Modifica carattere	`s[i] = c`	Troncamento con `s[i] = '\0'`
Scansione completa	`while (s[i] != '\0')`	Pattern universale per elaborazione testuale

Prossimi passi: Una volta padroneggiati questi fondamentali, esplora la libreria `<string.h>` con funzioni come `strlen`, `strcpy`, `strcat`, `strcmp` per operazioni più avanzate sulle stringhe.